BugForge: Constructing and Utilizing DBMS Bug Repository to Enhance DBMS Testing

Dawei Li †, Qifan Liu †, Yuxiao Guo, Jie Liang, Zhiyong Wu,
Chi Zhang, Jingzhou Fu, Haogang Mao, Zhenyu Guan, and Yu Jiang

26 arxiv

Introduction

A bug report serves as the basic entry or record within a bug repository, containing essential information about a specific bug.

- include a natural language description of the problem, version, environment information, logs, and typically an attached PoC.

A PoC (Proof of Concept) is a minimal artifact that demonstrates a bug or vulnerability.

- sequences of SQL statements that expose crashes, inconsistencies, or incorrect query results.

Basic Data:
#ID: 110804 **#Summary:** sql can't be logged in the slow query log.
#Status: Verified **#Severity:** S3 **#Component:** Logging #...

Whole Description:
When the variable min_examined_row_limit is set as a number larger than 0...
How to Repeat: **Set min_examined_row_limit larger than 0 such as 100.**
Execute a query as **"select count(*) from table_name"** which take longer time than long_query_time. The sql won't be logged in the slow query log.

Comments:
Reproduced as described. The rows_examined for the query “select count(*) from table_name” should not be calculated.

Fig. 1: MySQL Bug Report #110804

```
DELIMITER ^ ...
CREATE FUNCTION bug.test() RETURNS text
BEGIN
   SIGNAL SQLSTATE '45000';
   RETURN "returned from bug.test function";
END;^ ...
```

Fig. 2: PoC in MySQL bug report #102205

Motivation

Bug repositories are crucial in industry practice.

- They store bug-triggering information, especially PoCs, that reproduce failures.

Bug repositories are incomplete, unstructured, and hard to execute automatically.

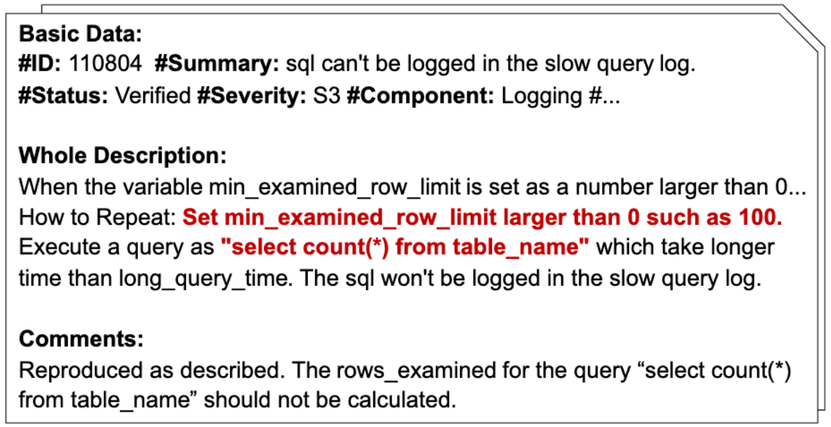
- Incomplete and ambiguous bug reports;
- Limited support for structured analysis;
- Siloed bug information across DBMSs.

This paper proposes *BugForge*, a framework that constructs standardized DBMS bug repositories and leverages them to generate high-quality test cases to enhance DBMS testing.

Challenge

C1 (难抽取): Mining bug information from **unstructured** and **evolving** DBMS bug reports that interleave SQL semantics with naturallanguage descriptions.

C2 (难重建): Restoring **executability** and **preserving semantic richness** to construct high-quality test cases that depend on specific database states.

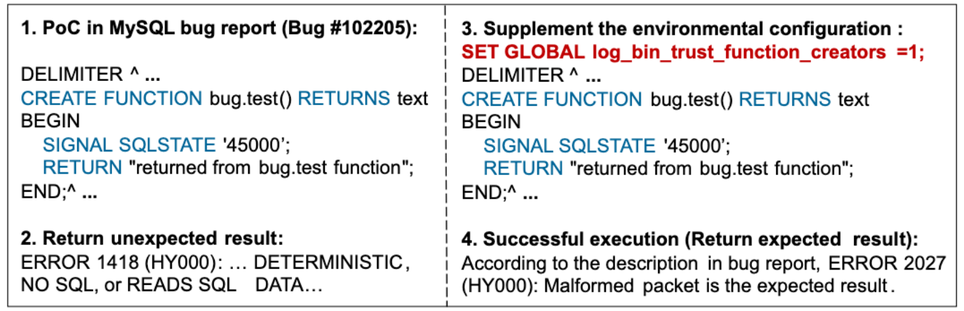


Basic Data:
#ID: 110804 **#Summary:** sql can't be logged in the slow query log.
#Status: Verified **#Severity:** S3 **#Component:** Logging #...

Whole Description:
When the variable min_examined_row_limit is set as a number larger than 0...
How to Repeat: **Set min_examined_row_limit larger than 0 such as 100.**
Execute a query as **"select count(*) from table_name"** which take longer time than long_query_time. The sql won't be logged in the slow query log.

Comments:
Reproduced as described. The rows_examined for the query "select count(*) from table_name" should not be calculated.

Fig. 1: MySQL Bug Report `#110804` contains a raw PoC that is difficult to extract automatically. This report complicates automated analysis, as the PoC is conveyed in prose instead of being explicitly presented in a distinct code block. The sequence of statements to reproduce the bug, must be manually reconstructed from the content of bug report.



1. PoC in MySQL bug report (Bug #102205):

```
DELIMITER ^ ...
CREATE FUNCTION bug.test() RETURNS text
BEGIN
  SIGNAL SQLSTATE '45000';
  RETURN "returned from bug.test function";
END;^ ...
```

2. Return unexpected result:
ERROR 1418 (HY000): … DETERMINISTIC, NO SQL, or READS SQL DATA…

3. Supplement the environmental configuration :

```
SET GLOBAL log_bin_trust_function_creators =1;
DELIMITER ^ ...
CREATE FUNCTION bug.test() RETURNS text
BEGIN
  SIGNAL SQLSTATE '45000';
  RETURN "returned from bug.test function";
END;^ ...
```

4. Successful execution (Return expected result):
According to the description in bug report, ERROR 2027 (HY000): Malformed packet is the expected result.

Fig. 2: MySQL Bug Report `#102205` misses the configuration required for successful execution. The bug is unreproducible because the bug report lacks the configuration information for the `log_bin_trust_function_creators` system variable, but it triggers the expected result after *BugForge* adapts the raw PoC.

Overview

The basic idea of BugForge is to use syntax-aware processing to extract raw PoCs and transform them into high-quality test cases through semantic-guided adaptation.

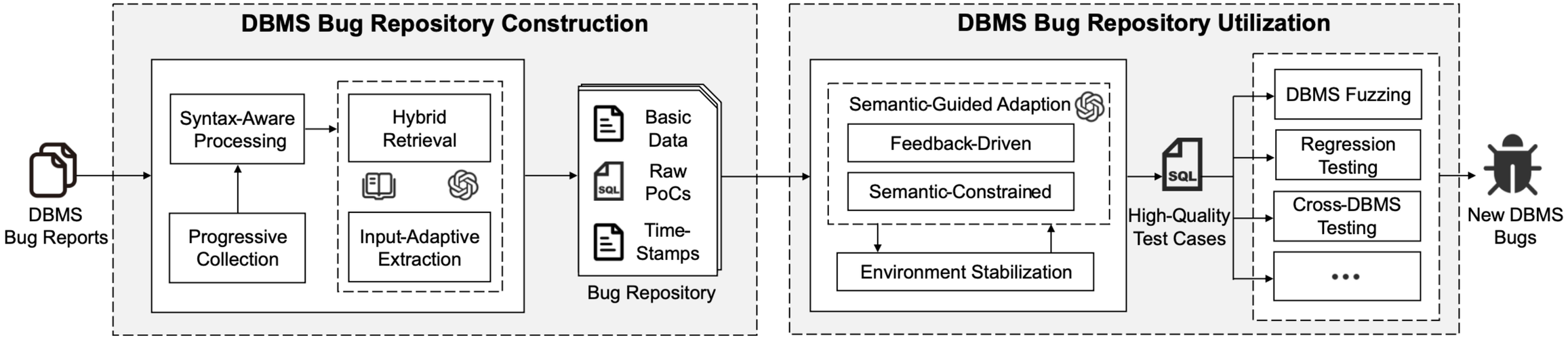


Fig. 3: **The overview design of *BugForge*.** *BugForge* is an end-to-end framework for DBMS bug repository construction and utilization. It progressively collects bug reports, applies syntax-aware processing and the input-adaptive LLM pipeline with RAG to extract raw PoCs and constructs a DBMS bug repository. Next, *BugForge* converts raw PoCs into high-quality test cases through semantic-guided adaptation under a stable environment and subsequently employs these cases in DBMS testing (e.g., fuzzing, regression, and cross-DBMS testing).

Overview

The basic idea of BugForge is to use syntax-aware processing to extract raw PoCs and transform them into high-quality test cases through semantic-guided adaptation.

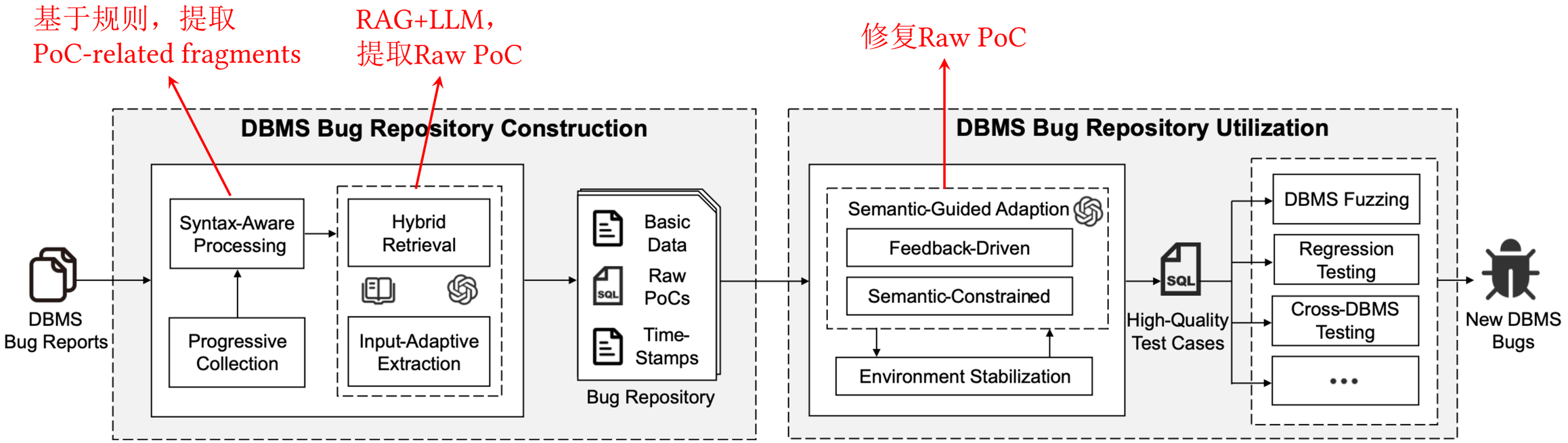


Fig. 3: **The overview design of *BugForge*.** *BugForge* is an end-to-end framework for DBMS bug repository construction and utilization. It progressively collects bug reports, applies syntax-aware processing and the input-adaptive LLM pipeline with RAG to extract raw PoCs and constructs a DBMS bug repository. Next, *BugForge* converts raw PoCs into high-quality test cases through semantic-guided adaptation under a stable environment and subsequently employs these cases in DBMS testing (e.g., fuzzing, regression, and cross-DBMS testing).

Syntax-Aware DBMS Bug Report Processing

PoC-related fragments: report fragments that may preserve procedural or semantic clues for constructing a raw PoC, but do not necessarily constitute a complete or directly high-quality raw PoC.

Algorithm 1: Syntax-Aware Bug Reports Processing

Input: DBMS bug report: $\mathcal{L}$.
Output: PoC-related fragments: $\mathcal{P}$, or **null**.

$\mathcal{S} \leftarrow$ an empty list of (index, content) tuples;
$i \leftarrow 0$;
while $i < |\mathcal{L}|$ **do**
  // Capturing Formatted SQL via Text Structure
  if $\mathcal{L}[i]$ *is a start-of-block tag (e.g., markdown)* **then**
    $j \leftarrow$ Find corresponding end tag from $i+1$;
    if j *is found* **then**
      Add $(i, \mathcal{L}[i+1:j])$ to $\mathcal{S}$; $i \leftarrow j$;
  // Capturing Fragmented SQL via SQL Feature
  else if $\mathcal{L}[i]$ *is a endpoint (e.g., semicolon)* **then**
    $j \leftarrow i$; $b \leftarrow 0$;
    while $j \geq 0$ **do**
      $b \leftarrow b + \text{Count}(\mathcal{L}[j],')') - \text{Count}(\mathcal{L}[j],'(')$;
      if *IsSQLSubject*$(\mathcal{L}[j])$ ***and*** $b \leq 0$ **then break**;
      $j \leftarrow j - 1$;
    Add $(j, \mathcal{L}[j:i+1])$ to $\mathcal{S}$;
  // Capturing Implicit SQL via Weight Calculation
  else if *CalculateScore*$(\mathcal{L}[i]) \geq \theta_{score}$ **then**
    Add $(i, [\mathcal{L}[i]])$ to $\mathcal{S}$;
  $i \leftarrow i + 1$;
return $\mathcal{P}$ if $\mathcal{P}$ is not **null** else **null**;

Given a DBMS bug report *L*, *BugForge* scans *L* using a prioritized **three-stage strategy** that progressively captures SQL code representations ranging from explicitly formatted blocks to fragmented or implicit statements.

阶段一：通过文本结构捕获格式化的 SQL
Markdown fenced block: ``` ... ```
Jira code block: {code:xxx} ... {code}

阶段二：通过SQL特性捕获分片化SQL
基于分号结尾的逆推
括号平衡约束等

阶段三：通过权重计算捕捉隐式 SQL

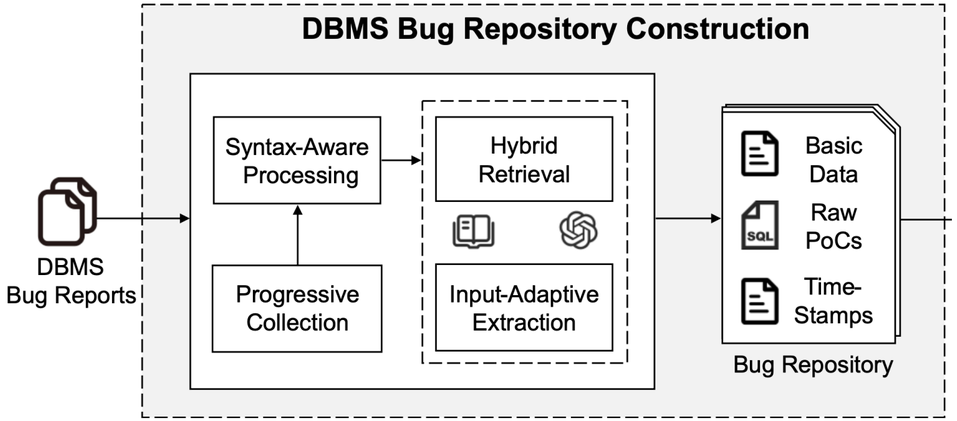
加分项
以分号结尾：+7
行首命中 SQL 主语：+18
命中方言关键字：每次 +3，最多 +14
...

减分项
绝对噪声，如 URL、十六进制地址
各种结构化噪声：Query OK、Records:
自然语言特征越多，扣分越多
...

Input-Adaptive Raw PoC Extraction

PoC-related fragments are incomplete, or semantically ambiguous.

To improve extraction accuracy while controlling context overhead, *BugForge* employs an LLM-based extraction framework with hybrid retrieval and adaptive input expansion.



(1) System Prompt: You are a PoC analysis and SQL extraction tool in DBMS.

(2) Extraction Prompt: You must follow this Three -Step Thinking process:
Step 1: Context-Guided Extraction: Extract the raw PoC from input content.
Step 2: On-Demand Expansion: Request extra bug report content only when needed.
Step 3: Self-Examination: Recheck whether the extraction follows these rules:

#R1: Preserve bug-triggering semantics. #R2: Remove irrelevant content and noise.
#R3: Do not invent unsupported details. #R4: Output N/A if the raw PoC is unextractable.

(3) Reference Prompt: (Hybrid-retrieved references from a dynamically evolving library)
Positive: (#Bug Summary #PoC-related Fragments) -> Chain-of-Thought -> raw PoC
Negative: (#Bug Summary #PoC-related Fragments) -> Chain-of-Thought -> N/A

(4) Task Prompt: Now, analyzing the current report with:
#Summary #PoC-related Fragments #Retrieved References. Request more content if needed.

Fig. 4: **An example prompt for LLM in raw PoC extraction.** It consists of four components: (1) System Prompt, which instructs LLM to analyze bug reports. (2) Extraction Prompt, which guides the LLM through extraction, on-demand context expansion, and self-examination. (3) Reference Prompt, which retrieves positive and negative exemplars from the library. (4) Task Prompt, which supplies the bug summary, raw PoC-related fragments, and retrieved references for the target report.

从动态维护的 reference library 中检索相似 exemplar。
检索方式是 dense retrieval (保证语义相似) + 关键词检索 (强调 DBMS 特有的词法特征，比如 SQL operator、error message)。

Input-Adaptive Raw PoC Extraction

Finally, after extracting a raw PoC, *BugForge* evaluates its quality in terms of **contextual consistency** and **structural completeness**.

- High-confidence cases are then distilled into reusable exemplars and inserted back into the RAG library.
- The validated raw PoC is then integrated with report metadata to construct the DBMS bug repository.

语义一致性得分方式：

1. 从参考 SQL 和候选 SQL 中抽取一组 semantic anchors，例如 objects/qualified_columns/functions/features/data_types等。
2. 对每个anchor分别计算： ***coverage*** = |ref ∩ cand| / |ref|,
   jaccard = |ref ∩ cand| / |ref ∪ cand|。
   每个anchor的得分: ***anchor_score*** = 0.7 * coverage + 0.3 * jaccard
3. 全局得分=加权平均所有anchor_score

#ID: BUG #18840
#Report Link: https://www.postgresql.org/message -id/... **#...**
#Summary: Segmentation fault in executing select unnest(array(oidvector)).
#Description: The total content of this bug... **#Comment：**The responses...
#CollectedDate: YYYY-MM-DD, reference timestamp for subsequent updates.
#Bug Type: (Crash√, Performance ×, Logic×, Unkown×)
#PoC-related Fragments: Report Fragments for constructing the raw PoC.

#Raw PoC:

```
CREATE TABLE my_table ( id SERIAL PRIMARY KEY, oidvector_col oidvector );
INSERT INTO my_table ( oidvector_col ) VALUES ( '12345 67890 54321' );
SELECT unnest ( ARRAY( SELECT oidvector_col )) from my_table;
```

Fig. 5: **An example case in *BugForge* repository.** *BugForge* normalizes heterogeneous DBMS bug reports into a structured representation. Each case includes: (1) basic metadata of the report, (2) summarized bug-type labels and the PoC-related fragments, (3) the extracted raw PoC used for downstream adaptation and DBMS testing.

Bug Repository Utilization

Object: convert raw PoCs into high-quality test cases and then leverages them for automated DBMS testing.

High-quality test cases: executability + semantic richness.

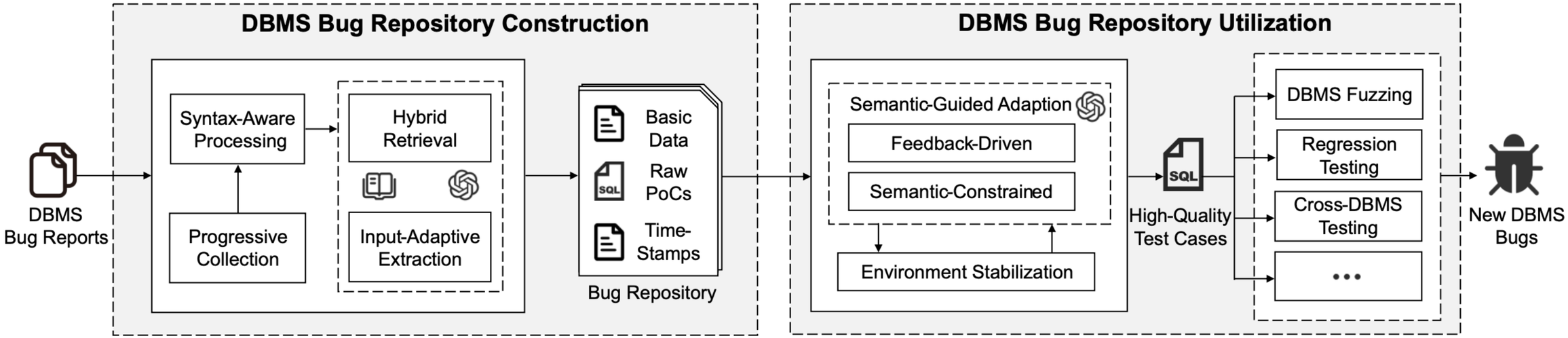


Fig. 3: **The overview design of *BugForge*.** *BugForge* is an end-to-end framework for DBMS bug repository construction and utilization. It progressively collects bug reports, applies syntax-aware processing and the input-adaptive LLM pipeline with RAG to extract raw PoCs and constructs a DBMS bug repository. Next, *BugForge* converts raw PoCs into high-quality test cases through semantic-guided adaptation under a stable environment and subsequently employs these cases in DBMS testing (e.g., fuzzing, regression, and cross-DBMS testing).

Semantic-Guided Test Case Adaptation

1. **Feedback-driven adaptation:** diagnoses execution-blocking issues through runtime feedback and repairs these.

根据人为构造的轻量级
知识库进行错误类型分类

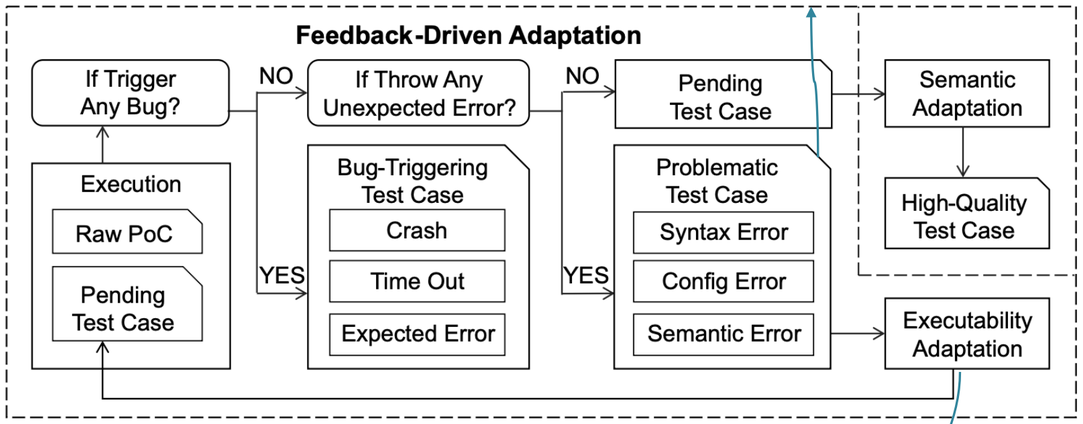


Fig. 6: **Process of Feedback-Driven Adaptation.** Each raw PoC is first executed and classified by its observed behavior. Cases with unexpected errors are further diagnosed using execution feedback and DBMS knowledge, and then repaired through LLM-guided executability adaptation.

```python
# --- 分类规则定义区 (保持不变) ---
CLASSIFICATION_RULES = {
    "Syntax Error": [
        'parse_error', 'syntax_error', 'wrong_name_for', 'dup_fieldname',
        'wrong_paramcount', 'illegal_grant', 'wrong_value_count', 'bad_sqlstate',
        'wrong_arguments', 'must_have_alias', 'unacceptable mutual recursion',
        'window specification', 'with column list', 'syntax', 'incorrect usage',
        'incorrect parameter', 'incorrect name', 'incorrect definition',
        'malformed'
    ],
    "Semantic Error": [
        'dup_entry', 'dup_key', 'no_such_table', 'bad_table_error', 'bad_field_error',
        'table_exists_error', 'no_referenced_row', 'row_is_referenced',
        'constraint_failed', 'unknown table', 'unknown column', 'table exists',
        'database exists', 'column count doesn', 'subquery returns more than 1 row',
        'ambiguous', 'unknown database', 'key column', 'doesn\'t exist',
        'not found', 'is not', 'already exists'
    ],
```

把 diagnosis 的结果（错误类型）+ runtime feedback（报错信息）
一起提供给 LLM，引导它补充缺失信息并修复执行阻塞问题

Semantic-Guided Test Case Adaptation

Semantic-constrained adaptation: uses semantic anchors captured from raw PoCs to guide adaptation towards results that do not lose the semantic richness of raw PoCs.

1. **Anchor match** is used to constrain the LLM to keep the core data dependencies of the original raw PoCs.
2. **Key coverage** aims to restrict LLM from modifying key SQL operations.
3. **Rewrite bound** is designed to prevent the LLM from excessively rewriting the original query structure.

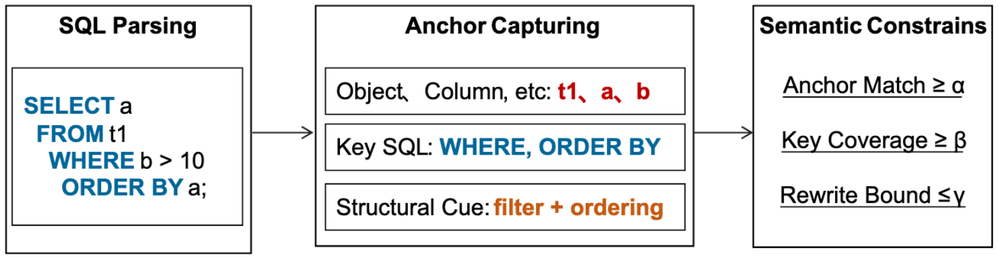


Fig. 7: **Process of Semantic-Constrained Adaptation.** *BugForge* first extracts semantic anchors from raw PoC, including data dependencies, SQL keywords, and structural cues, and organizes them into an anchor profile. Based on it, *BugForge* applies three constraints, namely anchor match, key coverage, and rewrite bound, to guide the LLM in preserving as much semantic richness in raw PoCs as possible during adaptation.

Rewrite Bound:

1. 把参考 SQL 和候选 SQL 做 token 化，计算它们的**序列相似度**：

   seq_similarity = SequenceMatcher(ref_tokens, cand_tokens).ratio()

2. 定义**改写强度**：

   rewrite_bound = 1 - seq_similarity

Environment Stabilization

To strictly guarantee execution independence while maximizing throughput, *BugForge* employs a statement-driven hierarchical impact assessment mechanism.

1. Parse input into atomic SQL statements.
2. Extract key semantics (verb, object, scope).
3. Map statements to risk levels：

   ① NONE（普通语句，对环境影响有限，不做额外补偿）

   ② RESTART（会改变运行时状态，但通常不需要彻底重建环境）

   ③ RESET（会污染全局环境、权限、插件、用户、数据库元信息，执行后需要强隔离恢复）

以 MariaDB 为例，见 mariadb/dry_run.py (line 560)：

- 如果出现这些语句，直接返回 `RESET`
  - `CREATE USER|ROLE`
  - `DROP USER|ROLE`
  - `ALTER USER|ROLE`
  - `GRANT`
  - `REVOKE`
  - `INSTALL PLUGIN|SONAME`
  - `UNINSTALL PLUGIN|SONAME`
  - `FLUSH PRIVILEGES`
- 如果没有命中上述 full-reset 语句，但命中：
  - `SET GLOBAL`
  - `SET PERSIST`

  那么返回 `RESTART`
- 否则返回 `NONE`

Evaluation

- RQ1（repository 的质量）: What are the characteristics and utilizability of the bug repository constructed by BugForge?
- RQ2（发现新 bug 的能力）: Can BugForge help discover new DBMS bugs?
- RQ3（与现有方法的对比）: How does BugForge compare with the state-of-theart DBMS testing tools?
- RQ4（test case 本身的价值）: How do high-quality test cases in BugForge compare with others in terms of DBMS testing?

Tested DBMSs: MySQL, PostgreSQL, MariaDB, MonetDB.

LLM-assisted stages: Gemini-2.5-flash for raw PoC extraction and Gemini-2.5-pro for the more complex adaptation tasks.

RQ1: Assessment of *BugForge* Bug Repository

Characteristic of Bug Repository.

- Collect all publicly available bug reports spanning from 1998 to 2026.

Utilizability of Bug Repository.

- Assess the usability by evaluating the coverage of SQL features and the number of adapted PoCs it contains.

TABLE I: Statistics of Bug Repository Built by *BugForge*

	MySQL	**MariaDB**	**PostgreSQL**	**MonetDB**	**Total**
Covered Time	2003-2026	2009-2026	1998-2026	2011-2026	–
Collected Reports	18,212	12,442	4,163	2,815	37,632
Extracted Raw PoCs	17,723	11,581	3,758	2,468	35,530
CVEs	2,055	147	364	31	2,597
high-quality Test Cases	16,112	11,426	3,616	2,444	33,598

TABLE II: Covered Data Types and Keywords in DBMSs

DBMS	**Data Type**			**Keyword**		
	Total	Covered	Rate	Total	Covered	Rate
MySQL	33	33	100.00%	754	666	88.33%
MariaDB	36	35	97.22%	683	616	90.19%
MonetDB	30	28	93.33%	301	268	89.04%
PostgreSQL	68	52	76.47%	491	452	92.06%
Total	**167**	**148**	**88.62%**	**2229**	**2002**	**89.82%**

RQ1: Assessment of *BugForge* Bug Repository

High-quality Test Cases Generated by BugForge.

- High-quality test cases refer to problematic raw PoCs that successfully pass both feedback-driven adaptation and semantic-constrained adaptation, meaning that they are executable and satisfy the semantic anchor constraints.

TABLE III: Number of High-quality Test Cases Adapted from Problematic Test Cases.

	MySQL	**MariaDB**	**PostgreSQL**	**MonetDB**	**Total**
Problematic Cases	12,504	4,523	1,911	292	**19,230**
Adaptable Cases	10,893	4,368	1,769	268	**17,298**
Ratio	87.12%	96.57%	92.57%	91.78%	**89.95%**

RQ2: Bugs Detected With *BugForge*

Three types of testing: DBMS fuzzing, DBMS regression testing, and cross-DBMS verification.

For *BugForge*, we use the configuration where *Griffin* is seeded with both built-in unit test cases and the high-quality test cases generated from *BugForge*.

TABLE IV: List of previously unknown bugs detected by DBMS testing with *BugForge* employed

#	DBMS	ID	Status	Component	Testing Method	Description
1	MySQL	S2310126	Confirmed	Function	DBMS Fuzzing	MySQL 9.4.0 crashes at Item_func_bit::val_int.
2	MySQL	S2310135	Confirmed	Function	DBMS Fuzzing	MySQL 9.4.0 crashes at Item_func::print_op.
3	MySQL	S2310142	Confirmed	Function	DBMS Fuzzing	MySQL 9.4.0 crashes at Item_func::walk.
4	MySQL	S2310157	Confirmed	Item	DBMS Fuzzing	MySQL 9.4.0 crashes Item_ref::walk.
5	MySQL	S2310161	Confirmed	Item	DBMS Fuzzing	MySQL 9.4.0 crashes at Item_ref::val_int.
6	MySQL	S2310174	Confirmed	Filed	DBMS Fuzzing	MySQL 9.4.0 crashes at Field::set_notnull.
7	MySQL	S2310188	Confirmed	Field	DBMS Fuzzing	MySQL 9.4.0 crashes at set_field_to_null_with_conversions.
8	MySQL	S2310190	Confirmed	Optimizer	DBMS Fuzzing	MySQL 9.4.0 crashes at Query_expression::optimize.
9	MySQL	S2310208	Confirmed	View	DBMS Fuzzing	Server crashes at Item_view_ref::check_column_privileges.
10	MySQL	# 119085	Reported	Server	Cross-DBMS Testing	FLUSH PRIVILEGES crash after table column drop.
11	MySQL	# 119086	Reported	Stored Routines	Cross-DBMS Testing	Server crash after VIEW-to-TABLE dependency change.
12	MariaDB	MDEV-37625	Confirmed	Server	DBMS Fuzzing	MariaDB 12.1.1-rc crashes at alloc_root.
13	MariaDB	MDEV-37628	Confirmed	Optimizer	DBMS Fuzzing	MariaDB 12.1.1-rc crashes at find_field_in_tables.
14	MariaDB	MDEV-37640	Confirmed	JSON/Server	DBMS Fuzzing	MariaDB 12.1.1-rc crashes at String::append.
15	MariaDB	MDEV-37641	Confirmed	Optimizer	DBMS Fuzzing	MariaDB 12.1.1-rc crashes at sub_select_postjoin_aggr.
16	MariaDB	MDEV-37646	Confirmed	Optimizer	DBMS Fuzzing	MariaDB 12.1.1-rc crashes at Item::save_decimal_in_field.
17	MariaDB	MDEV-37647	Confirmed	JSON/Optimizer/Server	DBMS Fuzzing	Server crashes at set_field_to_null_with_conversions.
18	MariaDB	MDEV-37648	Confirmed	Optimizer/Server	DBMS Fuzzing	Server crashes at Item_direct_view_ref::val_decimal.
19	MariaDB	MDEV-37768	Reported	Server	Cross-DBMS Testing	Server crash when mysql.procs_priv has an invalid structure.
20	MariaDB	MDEV-37671	Confirmed	Optimizer/Server	Regression Testing	Solved bug MDEV-32326 reappeared in 12.1.1-rc.
21	MariaDB	MDEV-37735	Confirmed	Sequences	Regression Testing	Solved bug MDEV-37172 reappeared in 11.8.4 and 12.1.1-rc.
22	MariaDB	MDEV-37734	Reported	Optimizer	Regression Testing	Solved bug MDEV-32308 reappeared in 11.8.4 and 12.1.1-rc.
23	MonetDB	# 7720	Confirmed	Server	DBMS Fuzzing	MonetDB Mar2025-SP2 server crashes at stmt_cond.
24	MonetDB	# 7721	Reported	Server	DBMS Fuzzing	MonetDB Mar2025-SP2 server crashes at stmt_replace.
25	MonetDB	# 7722	Fixed	Server	DBMS Fuzzing	MonetDB Mar2025-SP2 server crashes at rel_with_query.
26	MonetDB	# 7723	Reported	Server	DBMS Fuzzing	MonetDB Mar2025-SP2 server crashes at rel_has_freevar.
27	MonetDB	# 7724	Reported	Server	Regression Testing	Solved bug # 3009 reappeared in Mar2025-SP2 release.
28	MonetDB	# 7725	Fixed	Server	Regression Testing	Solved bug # 7482 reappeared in Mar2025-SP2 release.
29	MonetDB	# 7726	Reported	Server	Regression Testing	Solved bug # 7486 reappeared in Mar2025-SP2 release.
30	MonetDB	# 7727	Fixed	Server	Regression Testing	Solved bug # 7436 reappeared in Mar2025-SP2 release.
31	MonetDB	# 7728	Reported	Server	Regression Testing	Solved bug # 7488 reappeared in Mar2025-SP2 release.
32	MonetDB	# 7729	Confirmed	Server	Regression Testing	Solved bug # 7472 reappeared in Mar2025-SP2 release.
33	PostgreSQL	# 19055	Confirmed	Parser	DBMS Fuzzing	The count() aggregate is assigned the wrong agglevelsup.
34	PostgreSQL	# 19382	Fixed	Executor	DBMS Fuzzing	PostgreSQL 14-17.7 server crash at __nss_database_lookup.
35	PostgreSQL	# 19384	Reported	Parser	DBMS Fuzzing	PostgreSQL 17.7 server crashes at textout.

RQ3: Comparison of SOTA DBMS Testing Tools

The high-quality test cases generated from DBMS bug reports can guide testing toward deeper and broader execution paths than purely generator-based approaches.

TABLE V: Comparison of Typical DBMS Testing Tools.

DBMS	Covered Branches			Detected Bugs		
	SQLancer	SQLsmith	BugForge	SQLancer	SQLsmith	BugForge
MySQL	106,971	96,311	209,720	5	3	8
MariaDB	46,298	30,896	140,677	4	2	7
PostgreSQL	48,030	50,285	90,763	2	1	3
MonetDB	N/A	33,209	83,420	N/A	2	5
Total†	201,299	177,492	441,160	11	6	18

RQ4: Test Case Quality

U: built-in unit test cases from DBMS repositories.

B: *BugForge.*

Notably, the *BugForge* corpus contains only about 14.2 MB of total file content, compared with 100.3 MB for the unit-test corpus, corresponding to an 85.9% reduction.

TABLE VI: Covered Branches with different initial seeds.

DBMS	GRIFFIN^{U}	GRIFFIN^{B}	GRIFFIN^{U+B}	SQUIRREL^{U}	SQUIRREL^{B}	SQUIRREL^{U+B}
MySQL	182,709	192,710	209,720	147,936	159,794	175,794
MariaDB	114,029	134,595	140,677	89,115	100,188	109,318
PostgreSQL	73,273	85,862	90,763	59,253	63,235	74,828
MonetDB	32,990	79,797	83,420	N/A	N/A	N/A
Total	403,001	492,964	524,580	296,304	323,217	359,940
Increment	–	+22.32%	+30.17%	–	+9.08%	+21.48%

TABLE VII: Detected Bugs with different initial seeds.

DBMS	GRIFFIN^{U}	GRIFFIN^{B}	GRIFFIN^{U+B}	SQUIRREL^{U}	SQUIRREL^{B}	SQUIRREL^{U+B}
MySQL	4	6	8	3	5	6
MariaDB	3	5	7	2	4	5
PostgreSQL	1	2	3	1	1	2
MonetDB	2	5	5	N/A	N/A	N/A
Total	10	18	23	6	10	13
Increment	–	+80.00%	+130.00%	–	+66.67%	+116.67%

Conclusion

Problem: Bug reports contain valuable PoCs but are not directly usable.

Approach: BugForge builds and utilizes a unified bug repository.

Key Idea: Transform raw PoCs into executable, high-quality test cases.

Result: Improved coverage and discovered new bugs.